SPE-227721-MS

Solving Computational Fluid Dynamic Problems Using Quantum Computing

S. A. Al-Ogbi, Interdisciplinary Research Center for Intelligent Secure Systems, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia; M. Felemban, Interdisciplinary Research Center for Intelligent Secure Systems, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia / Computer Engineering Department and Information and Computer Science Department, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia

This paper was prepared for presentation at the Middle East Oil, Gas and Geosciences Show (MEOS GEO), Manama, Bahrain, 16 – 18 September 2025.

High-performance computing (HPC) refers to the use of advanced computational models and technologies to significantly enhance the performance of processing highly complex problems. HPC has evolved dramatically since the first supercomputer was commissioned in the 1960s. Nowadays, HPC is characterized by the pursuit of exascale computing, i.e., the ability to perform 10^{18} calculations per second. In November 2024, Lawrence Livermore National Laboratory, California announced the new El Capitan System as the most powerful exascale supercomputer, capable of performing over 1.742 exaFlops. Over the past few years, HPC platforms have evolved from CPU-centric architecture to GPU-accelerated systems. Such computing capability empowers researchers to tackle a diverse array of scientific applications across numerous domains, including stock market prediction, task scheduling, weather forecasting, gene sequencing, drug discovery, and Computational Fluid Dynamics (CFD) (Netto et al., 2018; Jamshed, 2015).

CFD is an applied science field that is concerned with the simulation and analysis of the physics of fluid flow using computational models and technologies. CFD finds extensive applications in diverse industries and scientific disciplines, including aircraft design optimization to facilitate the analysis of aerodynamic performance, drag reduction, and fuel efficiency enhancement (Moin and Apte, 2006). In addition, CFD plays a crucial role in oil reservoir simulation and recovery (Britto et al. 2020). Overall, CFD serves as a powerful tool for understanding and predicting fluid flow phenomena, enabling engineers and scientists to make informed decisions, optimize designs, and improve the performance of various systems and processes.

Recently, Quantum Computing has emerged as a new computing approach that leverages quantum physics phenomena such as superposition and entanglement to provide computational speedup over classical computers. With the rapid progress of quantum computing, it is projected that future HPC platforms are venturing into Quantum Processing Unit (QPU)-based systems to tackle previously unsolvable problems. Several researchers have studied the potential use of quantum computing in solving complex CFD problems (Gaitan, 2020; Jaksch et al. 2023). Quantum computing can offer several advantages in CFD applications. First, quantum computing can address the exponential growth of computational complexity associated with high-resolution and turbulent flow simulations. Accordingly, more accurate and detailed simulations can be performed. For example, Berry et al. (2014) presented quantum algorithms that can effectively handle large-

scale linear algebraic calculations, such as solving sparse matrix equations and eigenvalue problems, which are fundamental in CFD. Furthermore, quantum computers have the potential to accelerate optimization algorithms that facilitate faster design and optimization of fluid systems (Preskill, 2018). While quantum computing is still in its nascent stage, ongoing research in quantum hardware and algorithms demonstrates huge potential for future applications in solving complex CFD problems.

Introduction

High-performance computing (HPC) refers to the use of advanced computational models and technologies to significantly enhance the performance of processing highly complex problems. HPC has evolved dramatically since the first supercomputer was commissioned in the 1960s. Nowadays, HPC is characterized by the pursuit of exascale computing, i.e., the ability to perform 10^{18} calculations per second. In November 2024, Lawrence Livermore National Laboratory, California announced the new El Capitan System as the most powerful exascale supercomputer, capable of performing over 1.742 exaFlops. Over the past few years, HPC platforms have evolved from CPU-centric architecture to GPU-accelerated systems. Such computing capability empowers researchers to tackle a diverse array of scientific applications across numerous domains, including stock market prediction, task scheduling, weather forecasting, gene sequencing, drug discovery, and Computational Fluid Dynamics (CFD) (Netto et al., 2018; Jamshed, 2015).

CFD is an applied science field that is concerned with the simulation and analysis of the physics of fluid flow using computational models and technologies. CFD finds extensive applications in diverse industries and scientific disciplines, including aircraft design optimization to facilitate the analysis of aerodynamic performance, drag reduction, and fuel efficiency enhancement (Moin and Apte, 2006). In addition, CFD plays a crucial role in oil reservoir simulation and recovery (Britto et al. 2020). Overall, CFD serves as a powerful tool for understanding and predicting fluid flow phenomena, enabling engineers and scientists to make informed decisions, optimize designs, and improve the performance of various systems and processes.

Recently, Quantum Computing has emerged as a new computing approach that leverages quantum physics phenomena such as superposition and entanglement to provide computational speedup over classical computers. With the rapid progress of quantum computing, it is projected that future HPC platforms are venturing into Quantum Processing Unit (QPU)-based systems to tackle previously unsolvable problems. Several researchers have studied the potential use of quantum computing in solving complex CFD problems (Gaitan, 2020; Jaksch et al. 2023). Quantum computing can offer several advantages in CFD applications. First, quantum computing can address the exponential growth of computational complexity associated with high-resolution and turbulent flow simulations. Accordingly, more accurate and detailed simulations can be performed. For example, Berry et al. (2014) presented quantum algorithms that can effectively handle large-scale linear algebraic calculations, such as solving sparse matrix equations and eigenvalue problems, which are fundamental in CFD. Furthermore, quantum computers have the potential to accelerate optimization algorithms that facilitate faster design and optimization of fluid systems (Preskill, 2018). While quantum computing is still in its nascent stage, ongoing research in quantum hardware and algorithms demonstrates huge potential for future applications in solving complex CFD problems.

The objective of this paper is to investigate the feasibility of using existing quantum algorithms to accelerate classical algorithms for solving prominent CFD problems, specifically Navier-Stokes equations. We focus on the Quantum Amplitude Estimation Algorithms (QAEA) to address non-linear problems.

Background

In this section, we discuss prominent CFD problems and classical algorithms used to solve CFD problems addressing the challenges inherited in these algorithms. Furthermore, the section discusses the potential of quantum algorithms to overcome these challenges and enhance computational efficiency.

Computational Fluid Dynamics (CFD)

CFD is an applied science that utilizes computer simulations to predict the behavior of flowing fluids based on the conservation laws of mass, momentum, and energy. These predictions are typically studied under specific conditions, including flow geometry, fluid characteristics, and initial and boundary conditions. Various variables such as velocity, pressure, momentum, and temperature at specific locations and times of interest are the focus of these predictions. By knowing these variables, predictions can provide information about the flow behavior, such as pressure distribution, flow rate, and forces acting on the flow boundaries. Although numerous methods have been developed to simulate fluids, the results are not perfectly accurate due to errors from the discretization of the continuous system, approximation of input data, inaccurate initial and boundary conditions, or oversimplified modeling of complex physical phenomena (Landau et al., 2018).

Several CFD problems are modeled as linear and non-linear Partial Differential Equations (PDEs), which are used to provide a full description and overview of the physical properties of a fluid, such as velocity, pressure, and momentum. Solving these PDEs enable the researchers and engineers to analyze and predict the behavior of a flowing fluid in a wide range of applications, such as aerospace, fluid dynamics, and even in car designs (Chen et al., 2023; Bhatti et al., 2020). In the following section, we provide a brief introduction to a prominent CFD problem, namely Navier-Stokes equations.

Navier-Stokes Equations. Navier-Stokes equations are considered one of the most governing PDEs in CFD that model the fluid characteristics and the outside factors such as pressure gradients, and external forces (Temam, 1995; McLean, 2012). The Navier-Stokes existence and smoothness problem is listed among the seven-millennium problems (Blank, 2002). The Navier-Stokes system of equations consists of the continuity, momentum, and energy, in the three-dimensional space (x,y,z). It can be expressed, respectively, as follows (McLean, 2012):

The Continuity equation:

$$\frac{\partial \rho}{\partial t}+\frac{\partial(\rho u)}{\partial x}+\frac{\partial(\rho v)}{\partial y}+\frac{\partial(\rho w)}{\partial z}=0 \tag{1}$$

The Momentum equations, in (x,y,z), respectively:

$$\frac{\partial(\rho u)}{\partial t}+\frac{\partial(\rho u^2)}{\partial x}+\frac{\partial(\rho uv)}{\partial y}+\frac{\partial(\rho uw)}{\partial z}=-\frac{\partial \rho}{\partial x}+\frac{1}{Re}\left[\frac{\partial r_{xx}}{\partial x}+\frac{\partial r_{xy}}{\partial y}+\frac{\partial r_{xz}}{\partial z}\right], \tag{2}$$

$$\frac{\partial(\rho v)}{\partial t}+\frac{\partial(\rho uv)}{\partial x}+\frac{\partial(\rho v^2)}{\partial y}+\frac{\partial(\rho vw)}{\partial z}=-\frac{\partial \rho}{\partial y}+\frac{1}{Re}\left[\frac{\partial r_{xy}}{\partial x}+\frac{\partial r_{yy}}{\partial y}+\frac{\partial r_{yz}}{\partial z}\right], \tag{3}$$

and,

$$\frac{\partial(\rho w)}{\partial t}+\frac{\partial(\rho uw)}{\partial x}+\frac{\partial(\rho vw)}{\partial y}+\frac{\partial(\rho w^2)}{\partial z}=-\frac{\partial \rho}{\partial z}+\frac{1}{Re}\left[\frac{\partial r_{xz}}{\partial x}+\frac{\partial r_{yz}}{\partial y}+\frac{\partial r_{zz}}{\partial z}\right]. \tag{4}$$

The Energy equation:

$$\frac{\partial(E_t)}{\partial t}+\frac{\partial(uE_t)}{\partial x}+\frac{\partial(vE_t)}{\partial y}+\frac{\partial(wE_t)}{\partial z}=-\frac{\partial(\rho u)}{\partial x}-\frac{\partial(\rho v)}{\partial y}-\frac{\partial(\rho w)}{\partial z}-\frac{1}{Re{*}Pr}\left[\frac{\partial q_x}{\partial x}+\frac{\partial q_y}{\partial y}+\frac{\partial q_z}{\partial z}\right]+ \\ \frac{1}{Re}\left[\frac{\partial(u\tau_{xx}+v\tau_{xy}+w\tau_{xz})}{\partial x}+\frac{\partial(u\tau_{xy}+v\tau_{yy}+w\tau_{yz})}{\partial y}+\frac{\partial(u\tau_{xx}+v\tau_{xy}+w\tau_{xz})}{\partial z}\right] \tag{5}$$

where u,v, and w are the velocity terms, p is the pressure, ρ is the density, E_t is the total energy, Re is the Reynolds number, which describes the rate of the scaling of the inertia of the flow to the viscous forces, the q variables are the heat flux motifs, Pr is the Prandtl number, which describes the ratio of the viscous stresses to the thermal stresses, and τ variables are the c of stress tensor. The left-hand side of the momentum equations is known as the convection, which describes a physical process of ordered flow of a fluid, while the right-hand side represents the diffusion process, which is a physical process that occurs due to the random flow of the fluid.

Navier-Stokes equations have no analytical solution. However, approximate solutions can be computed iteratively based on the specific purpose of the situation. Classically, the iteration process can be achieved via different approaches, such as the conjugate gradient method (Hestenes and Stiefel, 1952), the generalized minimal residual method (Saad and Schultz, 1986), the successive over-relaxation (SOR) method (Young, 1954), or the multi-grid method (Wienands and Joppich, 2004).

Classical Algorithms for Solving CFD Problems

Over the past decades, a large number of techniques have been proposed to solve and simulate PDEs in CFD. This section discusses some of these methodologies, understand how they can be applied in such a perspective, and finally discuss the challenges in the face of these classical approaches.

Finite Difference Method. Using the Finite Difference Method to solve the CFD problem was first proposed by J. Von Neumann and R. D. Richtmyer (1950). Informally, the Finite Difference method works as the following. The derivatives in the PDE to be solved are approximated using finite differences, starting by discretizing the domain into grid points. Then the PDE is computed at the identified points discretely and the approximations are substituted in the PDE. Thus, the continuous PDE can be described as a system of equations, which associate the values at the grid points to their bordering points and other boundary conditions. The resultant system may be a system of linear or non-linear equations (Godunov et al., 1959).

There are many challenges in performing the Finite Difference method. One challenge is the appropriate handling of boundary conditions in order to provide an accurate presentation of the system. In addition, the utilization of the Finite Difference method enables the iterative property. Accordingly, PDEs can be solved using different techniques such as Gauss-Seidel, Jacobi, or more advanced methods. During the iteration process, the values at the grid points are updated based on the bordering points and the boundary conditions provided. Finally, the iteration is terminated once a convergence criterion is satisfied (Godunov et al. 1959). Another challenge is the control of solution accuracy by means of selecting the appropriate grid resolution. In particular, to extract accurate solutions using FDM, the grid must be sharp enough to capture the features needed properly in the simulations. Moreover, generating a convenient grid for complicated geometries could be difficult, especially when considering a 3-dimensional problem. Such cases require a huge number of grid points, which are computationally expensive in cost and memory. Finally, the FD approximation brings discretization errors into the system, which may also affect the accuracy and precision of the obtained solution (Strikwerda, 1984).

Finite Volume Method. Unlike the Finite Difference method, the Finite Volume method discretizes the domain into control volumes or cells (Versteeg and Malalasekera, 1995). From there, the PDEs are integrated over these volumes to construct the discrete volumes of the PDE. This approach is very useful in cases where conservation laws are involved. In the approximation step, the fluxes across the faces of the volumes are approximated, in which they represent the motion of the conserved quantities throughout the neighboring volumes. This conservation property produces simulations with higher sensitivity, which is very suitable for applications with irregular boundaries, thus, that makes FVM outperform FDM (Moukalled et al., 2016). Similar to the Finite Difference method, the Finite Volume method faces the same problem of generating the proper grids as well. Moreover, the Finite Volume method can suffer from numerical diffusion, especially when using low-order flux approximation schemes. Numerical diffusion can lead to the loss of fine-scale flow features and inaccuracies in the solution. High-order schemes and careful consideration of the numerical dissipation properties are employed to mitigate this issue. One more challenge is that when working with highly complicated geometries it might appear what's known as non-orthogonal grids. Special techniques are required to handle this type of grids, like interpolation schemes or by adding correction factors (Chung, 2002).

Spectral Methods. Another classical approach is the Spectral method which is used to represent CFD problems using spectral representations. It represents the flow variables in terms of a sum of the orthogonal basis functions in the spectral domain, e.g., Fourier expansion, Chebyshev polynomials, and Legendre polynomials. A very important advantage of using such spectral methods is the high-order accuracy they can provide, which makes them need a small number of grid points. This arises from the fact that this branch is very efficient when it comes to working on the smooth functions (Canuto et al., 2007). The spectral method excels in a wide range of problems, such as laminar, turbulent, incompressible, and compressible flows, boundary layer flows, and a lot more. Nevertheless, the Spectral method underperforms when solving complex geometries, unsteady problems, and problems with discontinuities. Many solutions were proposed to address these challenges, such as using hybrid approaches, where spectral methods are combined with other numerical techniques.

Quantum Algorithms for Partial Differential Equations

In this section, we review quantum algorithms that are proposed to solve CFD problems in the literature. However, we first start with a brief introduction to quantum computing. Unlike classical computers that use 0 or 1 bits to store and process data, quantum computing (due to quantum superposition) can simultaneously represent information in 0,1, or both using a quantum bit (qubit). In general, the linear combination can describe a qubit in superposition, such that: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, in which α and β are two complex numbers that their squares must sum up to1, $\alpha^2 + \beta^2 = 1$. When a qubit is measured, the qubit state is reduced into one of the states, either $|0\rangle$ with a probability of α^2 or $|1\rangle$ with a probability of β^2. To manipulate the qubits, quantum gates act on them and evolve and change their states with time. An *n*-qubit quantum operation can be expressed as $2_n \times 2_n$ unitary matrices (U) in such a manner that makes a return to the identity matrix $UU^\dagger = U^\dagger U = I$.

The tensor product is used to describe a "mix" state of qubits to create bigger quantum states. For example, say that the 1-qubit two states $|\psi\rangle$ and $|\phi\rangle$, has a tensor product of $|\psi\rangle\otimes|\phi\rangle$. The resultant quantum state is a joint of the four states: $|\phi\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$ where $\alpha^2 + \beta^2 + \gamma^2 + \delta^2=1$ (Al-Ogbi et al., 2022). In addition to other properties, superposition empowers quantum computers and allows executing computations in parallel which can lead to exponential speedup in solving problems over classical computers (Nielsen and Chuang, 2010). Similar to classical algorithms, quantum algorithms are computational algorithms that are designed to be run on quantum computers. By taking advantage of the principles of quantum mechanics, quantum algorithms are assumed to solve problems in less time, and/or computational complexities compared to classical algorithms (Al-Ogbi et al., 2022; Nielsen and Chuang, 2010; Ladd et al. 2010). In the following subsections, we illustrate one prominent quantum algorithm in detail to understand how it can be applied in the field of CFD simulations.

Quantum Amplitude Estimation Algorithm

Quantum amplitude Estimation (QAE) algorithms are a class of quantum algorithms that target estimating the amplitude of a specific state with high precision. Such algorithms have several applications like quantum optimization, machine learning, and simulation (Ladd et al. 2010). It was reported that quantum amplitude estimation/amplification can provide an exponential speedup over classical solutions (Brassard et al. 2000). QAE algorithm can be executed, by initializing a quantum state, usually referred to as $|\psi\rangle$, which is a superposition of all possible states. Also, in this step, one can determine the target state that we want to estimate its amplitude. Then, amplitude amplification techniques are applied to increase the probability of measuring the target state. After, quantum phase estimation is applied to estimate the phase associated with the target state's amplitude. The phase estimation process involves applying controlled operations and quantum Fourier transforms to extract the phase information. Finally, the circuit can measure the quantum state, to extract the outcome corresponding to the estimated phase value. This estimated phase value is then used to calculate the estimated amplitude of the target state.

The quantum circuit for QAE in Fig. 1 illustrates how QAE works. In the figure, we can observe the unitary operator Q:

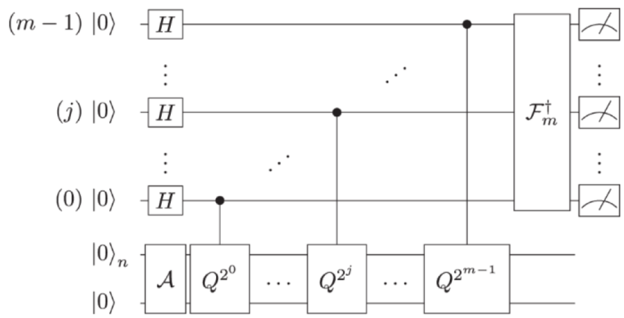

Figure 1—Quantum amplitude estimation circuit (Grinko et al. 2021).

$$\mathcal{Q} = -\mathcal{A}\mathcal{S}_0\mathcal{A}^{-1}\mathcal{S}_\chi$$

where Q^j means it is applied j times, to employ the iterative concept and operator A being any quantum operator/algorithm, S_X is the negation operator that changes the sign of the amplitudes if $|x\rangle \mapsto -|x\rangle$ only if $X(x) = 1$, and S_0 is the operator that changes the sign of the amplitude only if the state is zero (Grinko et al. 2021).

Several researchers proposed to use QAE algorithm in solving CFD problems and showed a quadratic (or exponential) speedup compared to classical algorithms, depending on the size of the problem. Gaitan et al. (2020) proposed to combine the Quantum Amplitude Estimation algorithm to solve Navier-Stokes Equations (Gaitan, 2020). They proposed the significance of reducing the system of partial differential equations (PDEs) to a system of ordinary differential equations (ODEs) by discretization. Then, the Quantum Amplitude Estimation (QAE) algorithm is used to approximate a solution to the ODEs system. Upon testing their proposal on a steady-state inviscid, compressible fluid that's passing through a converging-diverging nozzle, with the presence and absence of a shockwave, they found a great consistency between the obtained results and the exact solutions with and without a shockwave. They have reported a quantum quadratic/exponential speedup over the existing classical algorithms. Moreover, they illustrated a speedup in the rough/non-smooth flows, which is typically computationally expensive as it's a difficult case.

Oz et al. (2021) investigated the potentials of using the QAE algorithm in CFD simulations by proposing a Quantum CDF solver, and adoption of the methodology proposed in (Gaitan, 2020) to solve two cases of Burgers' Equation, considering the absence and presence of a shockwave (Oz et al., 2022). In both cases (smooth flow and a travelling shockwave), they have found an outstanding match between the acquired results and the exact solutions, with a noticeable quantum speedup (quadratic/exponential) over the classical algorithms that have been used in the same context.

A follow-up work by Oz et al. (2023) was done by adapting the methodology in (Gaitan, 2020) to propose a quantum solver for PDEs with the integration of Chebyshev points (Oz et al., 2023). In their work, three different PDEs were used to carry out simulations: a generic ODE, the Heat equation, and convection-diffusion equation. When testing the proposed technique for the three types of equations, they observed an improvement in the accuracy up to two-orders, in 100 times faster time of solving.

Methodology

Problem Formulation

Adopting the approach proposed in (Gaitan, 2020), Navier-Stokes Equations in one spatial dimension can be expressed as follows:

$$\frac{\partial \rho}{\partial t}+\frac{\partial}{\partial x}(\rho v)=0, \tag{6}$$

$$\frac{\partial}{\partial t}(\rho v)+\frac{\partial}{\partial x}(\rho v2+p-r)=0, \tag{7}$$

And

$$\frac{\partial}{\partial t}\left[\rho\left(e+\frac{v2}{2}\right)\right]+\frac{\partial}{\partial x}\left[\rho v\left(e+\frac{v2}{2}\right)+pv-\tau+\kappa\frac{\partial T}{\partial x}\right]=0 \tag{8}$$

Where v represents the local flow velocity, $\tau=(2\mu+\lambda)\frac{\partial v}{\partial x}$ represents the Newtonian viscous stress with first (second) viscosity coefficients $\mu(\lambda)$, κ represents the thermal conductivity, and $e(\rho,T)$ represents the local internal energy.

In some specific cases of flow, the local internal energy and the pressure $e(\rho,T)$ and $p(\rho,T)$, respectively) can be identified as the quantities that vary with the mass and temperature of the system, and therefore, they could be taken off the Navier-Stokes system. By reducing Equations 6, 7, and 8, they can be computed for the unknown variables: $v(x,t)$, $\rho(x,t)$, and $T(x,t)$.

After conservation, the Navier-Stokes Equations are expressed as:

$$\frac{\partial U}{\partial t}+\frac{\partial F\left[U,\frac{\partial U}{\partial x}\right]}{\partial x}=J[U] \tag{9}$$

In which the three-component column vectors are represented by flow variables $U=(U_1,U_2,U_3)_T$, fluxes $F=(F_1,F_2,F_3)T$, and sources $J=(J_1,J_2,J_3)_T$, and T is the transpose operation.

The spatial derivatives in Equation 9 can be estimated using algebraic methods. This transforms the Navier-Stokes equations from a non-linear partial differential system into a non-linear ordinary differential system that can be stated as:

$$\frac{dU}{dt}=f(U) \tag{10}$$

After examination, they concluded that only $F\left[U,\frac{\partial U}{\partial x}\right]$ and, thus, $f(U)$, consist of expressions that depend on the viscosity. From there, the task to compute a bounded function $I(j,t)$ to approximate the exact solution $U(j,t)$ for $0\le t\le T$, with respect to the initial condition $I(j,0)=U(j,0)=U_0\,(j)$. At the beginning, the interval $[0,T]$ is divided into n smaller intervals ($I_i=[t_i,t_{i+1}]$) of period $h\ =\frac{T}{n}$, and then, those sub-intervals are also divided into further intervals $N_k=n^{(k-1)}(I_i^I=[t_i^I,t_i^{(I+1)}])$ of period $\hbar=\frac{h}{N_k}$.

Finally, we can conclude the function where the quantum algorithm is used, such that:

$$y_{i+1}(j)=y_i(j)+N_k\sum_{I=0}^{N_k-1}\frac{\hbar}{N_k}\int_0^1 du f\left(I_{i,I}\ \ (j,\ \ u)\right) \tag{11}$$

for $0\le I\le n$-1, where $y_i\,(j)\approx U(j,t_i)$ is the exact solution, and I_i,I is the initial condition for the approximated solution. As proposed in (Gaitan, 2020), the right-hand side, f specifically, in Equation 11 is the only part that needs a quantum computer to be estimated.

Implementation of Quantum Amplitude Estimation

To encode function *f* into a quantum circuit, the circuit in Fig. 1 is can be decomposed into the following operators, as illustrated in Fig. 2 below:

$$\mathcal{A} = \left(\bigotimes \mathcal{I}\right)\mathcal{R} \tag{12}$$

Where P is the operator that encodes the probability distribution $p(x) = \frac{1}{2^n}$, I is the identity operator that acts of the last qubit, and . is the operator that encodes the function to be estimated within a specific interval.

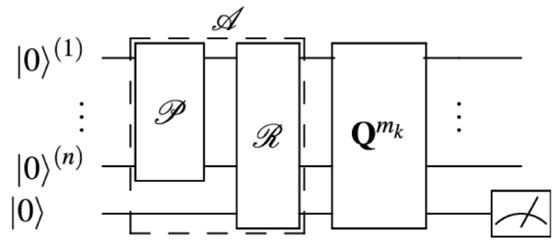

Figure 2—Closer snapshot to the Canonical QEA circuit (Suzuki et al., 2020).

For simplicity, we assumed:

$$f = \int_0^{100} \sin^2(x)dx, \tag{13}$$

And

$$f = \int_0^{100} \cos^2(x)dx \tag{14}$$

Encoding the functions in Equations 13 and 14 into the operator R was done using CRX- and CRY-gates. Then, for the probability operator P, Hadamard (H) gate was used to provide the system with a uniform probability. The uniform probability was deployed here to provide a uniform chance for all quantum states to be estimated. This reduces the chances of omitting any information that might be important.

The implementation of operator A for Equations 13 and 14 was conducted using various publicly available quantum computing platforms, focusing on *Qibo library*, by Zapata Computing. Zapata Computing is quantum software that develops industrial generative AI for business solutions. Zapata emerged in 2017, becoming an important competitor in the field of quantum computing. One of their contributions is the Qibo open-source platform. In order to fulfill this task, it was programmed on Python $v3.11$ using Qibo $v0.2.7$.

Implementationof $\boldsymbol{f\int_0^{100} sin^2(x)dx}$ ***Circuit.***

Encoding Equation 13 into operator A of the quantum circuit was conducted using 7 qubits and a series of H- and Controlled-RX gates, as shown in Fig. 3 below.

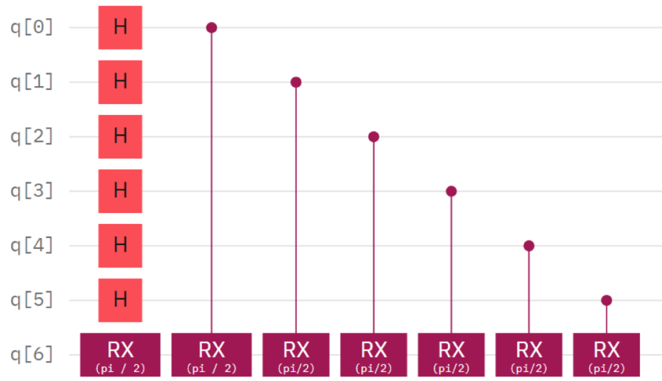

Figure 3—Implementation of a 7-qubit operator A circuit for Equation 13.

Then, encoding Q^{mk} to the quantum circuit results in the full QAEA circuit illustrated in Fig. 4.

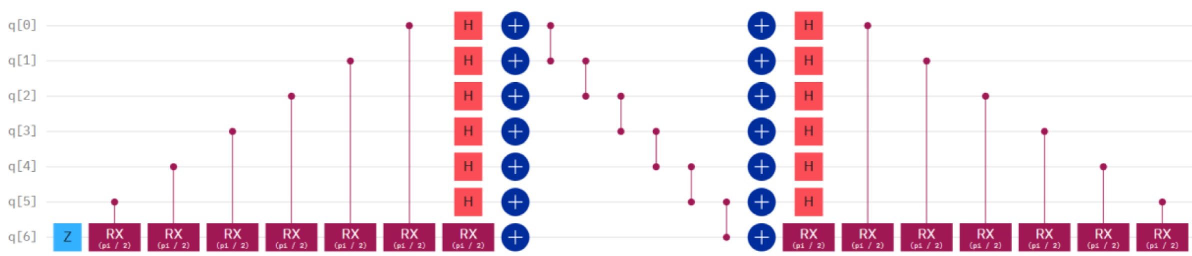

Figure 4—Implementation of a 7-qubit QAEA circuit for Equation 13.

To execute the estimation, significance level was set to α = 0.05, precision level was set to $\epsilon = 0.005$, along with Chernoff bounds. The results were obtained using a 7-qubit quantum circuit of depth 18, with 42 gates.

$$\boldsymbol{Implementation of \int_0^{100} cos^2(x)dx\ \ Circuit.}$$

Implementing this circuit was done similarly to the previous one, with some changes in the rotational gates, as CRY-gate was used, as shown in Fig. 5,

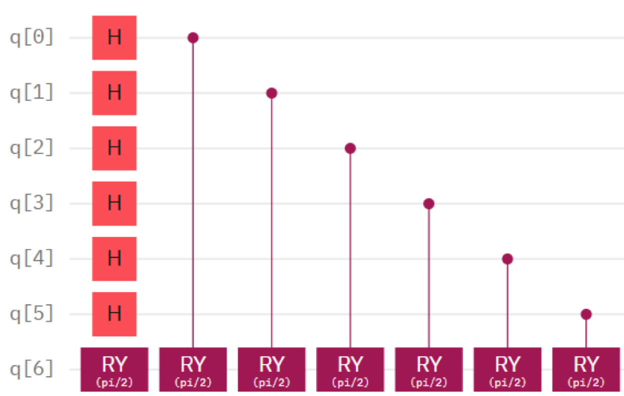

Figure 5—Implementation of a 7-qubit implementation of operator A circuit for Equation 14.

And Fig. 6, below.

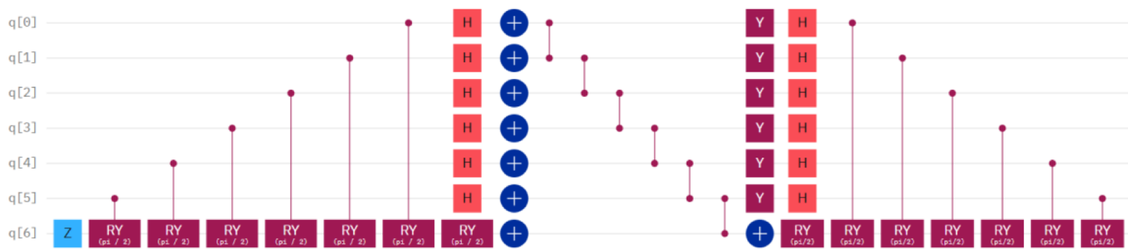

Figure 6—Implementation of a 7-qubit circuit of QAEA for Equation 14.

Implementation of Quantum Algorithms Optimization Techniques

In this task, an extensive review of quantum circuit optimization techniques was conducted. Various optimization techniques were implemented to make the quantum circuit in Fig. 4 more practical when running on existing quantum computing platforms. These techniques included gate-merging, gate replacement, and circuit fusion to minimize the depth of the quantum circuit (Efthymiou et al., 2021).

$$\boldsymbol{Optimization\ of} \int_0^{100} \boldsymbol{sin^2(x)dx\ \ Circuit.}$$

Gate Merging Optimization Technique. Operator $\boldsymbol{Q^{mk}}$ consists of a combination of multiple operators represented by Z, X, and H-gates, as illustrated on the left-hand side of Fig. 4. We were able to merge the XZX sequence of gates into a single H gate, as shown in Fig. 7.

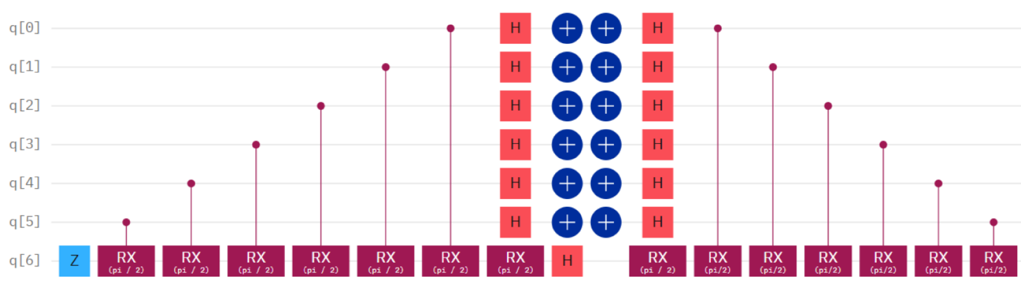

Figure 7—Implementation of a 7-qubit circuit using gate-merging technique for Equation 13.

Performing the estimation under the using the same parameters showed that this technique reduced the circuit depth down to 17, using total number of gates of 40.

Gate Fusion (Grouping) Optimization Technique. In this step, gates that can be performed in one step were grouped, and the estimation was re-executed with the same parameters. This technique was applied twice on the circuits in Fig. 4 and Fig. 7. The technique has reduced the circuit in Fig. 4 to be of depth 13, with the gates fused, as Fig. 8 below shows.

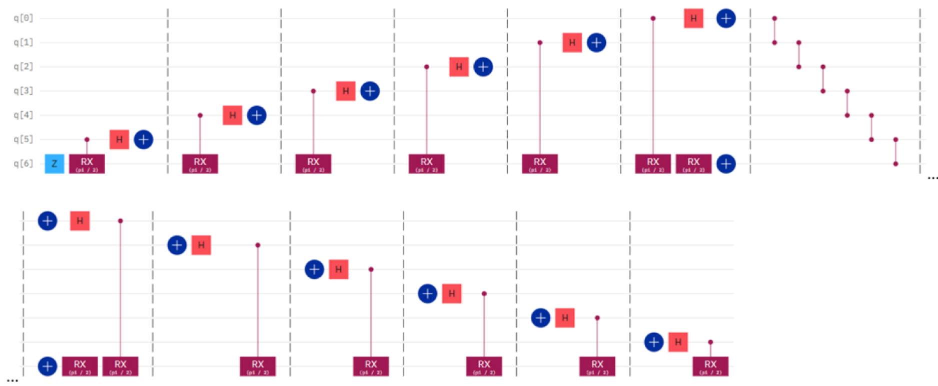

Figure 8—Implementation of a 7-qubit circuit using fusion technique for quantum circuit in Fig. 4.

While the circuit in Fig. 7 was reduced to a depth of 11, as Fig. 9 illustrates.

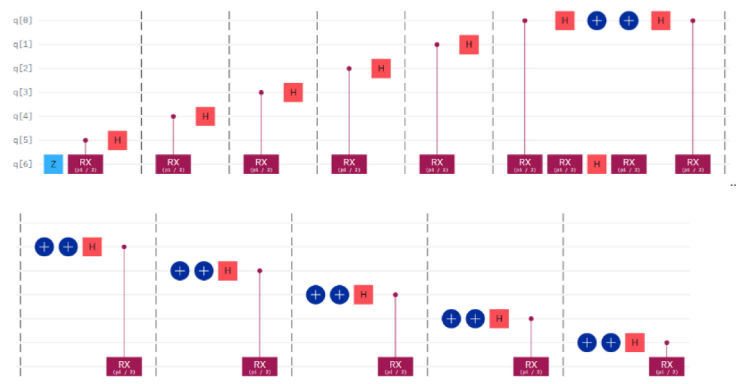

Figure 9—Implementation of a 7-qubit circuit using Fusion technique for quantum circuit in Fig. 8.

$$\textbf{\textit{Optimization of}} \int_0^{100} \cos^2(x)dx \textbf{\textit{ Circuit.}}$$

Gate Merging Optimization Technique. Applying the gate-merging optimization technique to the quantum circuit in Fig. 6 resulted in the following circuit, displayed in fig. below. The resultant quantum circuit depth was of 17, using 40 quantum gates.

Gate Fusion (Grouping) Optimization Technique. Then, performing the gate fusion optimization technique on circuits Fig. 6 and Fig. 10 resulted in circuits illustrated below.

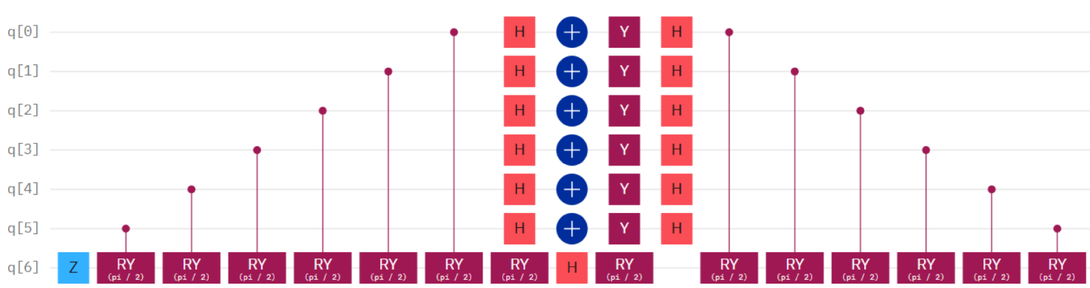

Figure 10—Implementation of a 7-qubit circuit using gate-merging technique for Equation 14.

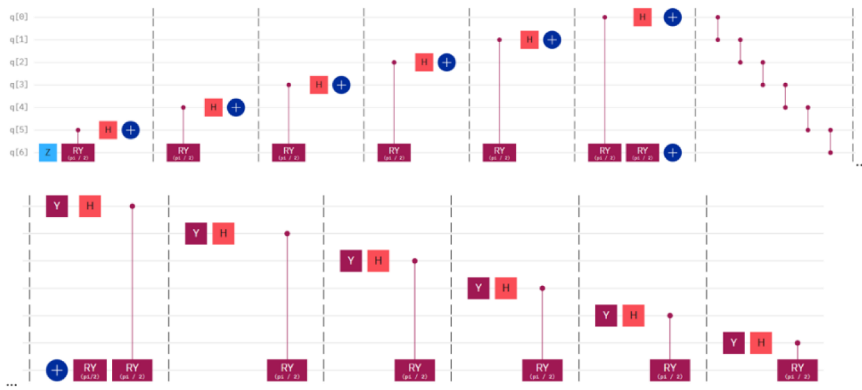

Figure 11—Implementation of a 7-qubit circuit using gate fusion technique for quantum circuit in Fig. 6.

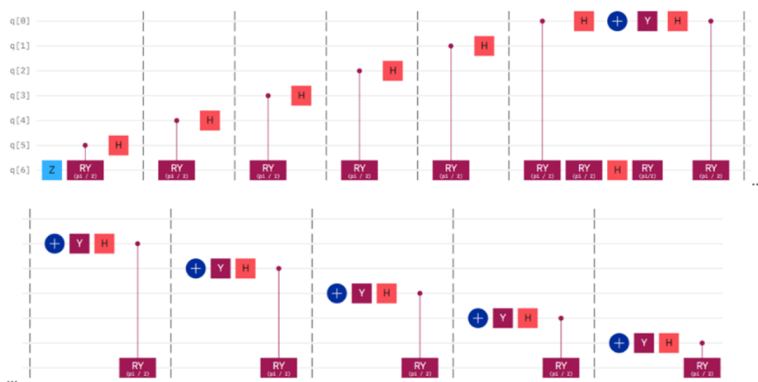

Figure 12—Implementation of a 7-qubit circuit using gate fusion technique for quantum circuit in Fig. 10.

In addition, to ensure that the number of qubits do not be a disruptive factor, the code was programmed to increase the number of qubits automatically upon reaching the higher bound of Chernoff, 0.005.

Results and Discussion

Estimating Equation 13 using the various versions of QAEA showed excellent agreement with the classical results obtained from Python calculator using *Scipy* and *Numpy* libraries. The estimation was executed 10 times for each case, as shown in Fig. 13, with average computational errors of 0.0042 and 0.004, respectively.

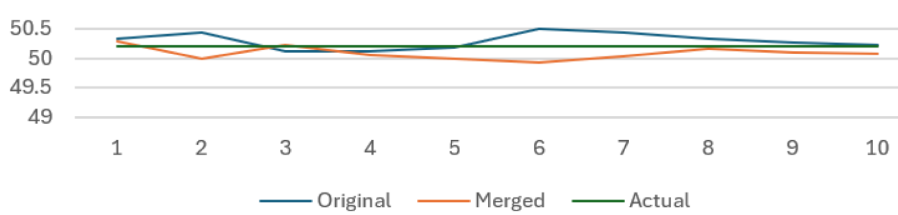

Figure 13—A performance comparison of estimating Equation 13. Comparison was conducted using the original and merged approaches against the classical one.

However, estimating Equation 14 using the quantum algorithm exhibited some deviations in the results from the classical one, as Fig. 14 illustrates. This case showed an average computational error of 0.0044 and 0.0038, respectively.

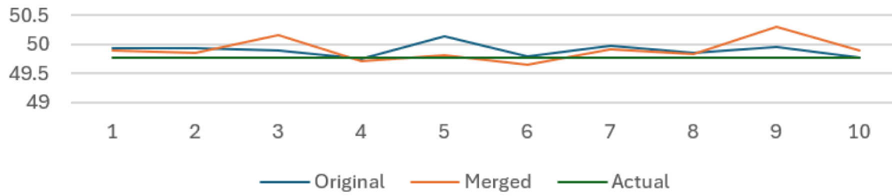

Figure 14—A performance comparison of estimating Equation 14. Comparison was conducted using the original and merged approaches against the classical one.

Conclusion

In conclusion, we have found that using the Iterative Quantum Amplitude Estimation Algorithm is highly efficient for estimating periodic functions, as demonstrated by the excellent agreement with the classical results. This validates the potential of quantum algorithms, particularly QAEA, in solving non-linear partial differential equations when simplified into a mean-value problem.

This algorithm showed it superior performance over the classical methods in the integration of $\mathbf{sin^2}\boldsymbol{x}$; however, challenges remain with $\mathbf{cos^2}\,\boldsymbol{x}$. These findings highlight the significance of quantum computing algorithms in advancing research in solving such equations, paving the way for improved simulations and solutions to the more complicated problems in fluid dynamics, including oil reservoir modeling, gas simulations, and aircraft design.

References

Al-Ogbi, S., Ashour, A., & Felemban, M. (2022, December). Quantum image classification on NISQ devices. In *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)* (pp. 1–7). IEEE.

Berry, D. W., Childs, A. M., Cleve, R., Kothari, R., & Somma, R. D. (2014). Exponential improvement in precision for simulating sparse Hamiltonians. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing* (pp. 283–292).

Bhatti, M. M., Marin, M., Zeeshan, A., & Abdelsalam, S. I. (2020). Recent trends in computational fluid dynamics. *Frontiers in Physics*, **8**, 593111.

Blank, B. E. (2002). The Millennium Problems: The Seven Greatest Unsolved Mathematical Puzzles of Our Time. *Notices of the Ams*, **50**(8).

Brassard, G., Hoyer, P., Mosca, M., & Tapp, A. (2000). Quantum amplification and estimation. arXiv preprint quant-ph/0005055.

Britto, A. F., Vivas, C. S., Almeida, M. P., da Cunha Lima, I. C., & da Cunha Lima, A. T. (2020). Multiphase flow mobility impact on oil reservoir recovery: An open-source simulation. *AIP Advances*, **10**(3).

Canuto, C., Hussaini, M. Y., Quarteroni, A., & Zang, T. A. (2007). *Spectral methods: evolution to complex geometries and applications to fluid dynamics*. Springer Science & Business Media.

Chen, L. W., & Ishak, I. A. (2023). CFD Analysis on the Aerodynamic Characteristics of Small-Sized Car Vehicles Under the Influence of Steady Crosswind. *Journal of Emerging Technologies and Industrial Applications*, **2**(1).

Chung, T. J. (2002). *Computational fluid dynamics*. Cambridge university press.

Efthymiou, S., Ramos-Calderer, S., Bravo-Prieto, C., Pérez-Salinas, A., García-Martín, D., Garcia-Saez, A., ... & Carrazza, S. (2021). Qibo: a framework for quantum simulation with hardware acceleration. *Quantum Science and Technology*, **7**(1), 015018.

Gaitan, F. (2020). Finding flows of a Navier–Stokes fluid through quantum computing. *npj Quantum Information*, **6**(1), 61.

Godunov, S. K., & Bohachevsky, I. (1959). Finite difference method for numerical computation of discontinuous solutions of the equations of fluid dynamics. *Matematičeskij sbornik*, **47**(3), 271–306.

Grinko, D., Gacon, J., Zoufal, C., & Woerner, S. (2021). Iterative quantum amplitude estimation. *npj Quantum Information*, **7**(1), 52.

Hestenes, M. R., & Stiefel, E. (1952). Methods of conjugate gradients for solving linear systems. *Journal of research of the National Bureau of Standards*, **49**(6), 409–436.

Jaksch, D., Givi, P., Daley, A. J., & Rung, T. (2023). Variational quantum algorithms for computational fluid dynamics. *AIAA journal*, **61**(5), 1885–1894.

Jamshed, S. (2015). *Using HPC for Computational Fluid Dynamics: A guide to high performance computing for CFD engineers*. Academic Press.

Ladd, T. D., Jelezko, F., Laflamme, R., Nakamura, Y., Monroe, C., & O'Brien, J. L. (2010). Quantum computers. *nature*, **464**(7285), 45–53.

Landau, L. D., & Lifshitz, E. M. (2013). *Course of theoretical physics*. Elsevier.

McLean, D. (2012). Continuum fluid mechanics and the Navier-Stokes equations. *Understanding Aerodynamics: Arguing from the Real Physics*, 13–78.

Moin, P., & Apte, S. V. (2006). Large-eddy simulation of realistic gas turbine combustors. *AIAA journal*, **44**(4), 698–708.

Moukalled, F., Mangani, L., & Darwish, M. (2016). *The finite volume method* (pp. 103–135). Springer International Publishing.

Netto, M. A., Calheiros, R. N., Rodrigues, E. R., Cunha, R. L., & Buyya, R. (2018). HPC cloud for scientific and business applications: taxonomy, vision, and research challenges. *ACM Computing Surveys (CSUR)*, **51**(1), 1–29.

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information*. Cambridge university press.

Oz, F., San, O., & Kara, K. (2023). An efficient quantum partial differential equation solver with Chebyshev points. *Scientific Reports*, **13**(1), 7767.

Oz, F., Vuppala, R. K., Kara, K., & Gaitan, F. (2022). Solving Burgers' equation with quantum computing. *Quantum Information Processing*, **21**(1), 30.

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, **2**, 79.

Saad, Y., & Schultz, M. H. (1986). GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on scientific and statistical computing*, **7**(3), 856–869.

Strikwerda, J. C. (1984). Finite difference methods for the Stokes and Navier–Stokes equations. *SIAM Journal on Scientific and Statistical Computing*, **5**(1), 56–68.

Suzuki, Y., Uno, S., Raymond, R., Tanaka, T., Onodera, T., & Yamamoto, N. (2020). Amplitude estimation without phase estimation. *Quantum Information Processing*, **19**, 1–17.

Temam, R. (1995). Navier–Stokes equations and nonlinear functional analysis. *Society for industrial and applied mathematics*.

Versteeg, H. K., & Malalasekera, W. (1995). Computational fluid dynamics. *The finite volume method*, 1–26.

VonNeumann, J., & Richtmyer, R. D. (1950). A method for the numerical calculation of hydrodynamic shocks. *Journal of applied physics*, **21**(3), 232–237.

Wienands, R., & Joppich, W. (2004). Practical Fourier analysis for multigrid methods. Chapman and Hall/CRC.

Young, D. (1954). Iterative methods for solving partial difference equations of elliptic type. *Transactions of the American Mathematical Society*, **76**(1), 92–111.